ELEC0028 Advanced Digital Design Coursework Assignment

Table of Contents

Task 1.1 2

Task 1.2 – Writing SystemVerilog description of the sub-blocks 3

Program counter module 3

SystemVerilog code 3

Extend module 4

SystemVerilog code 4

Control Unit Module 4

SystemVerilog Code 4

ALU Module 8

SystemVerilog Code 8

Task 1.3 – Simulations of sub-blocks using testbenches 9

Program counter Testbench 9

Simulation results 10

Extend Testbench 11

Simulation Results 11

Control Unit Testbench 12

Simulation Results 14

ALU Testbench 15

Simulation Results 16

Task 1.4 – Completing top-level RISC-V SystemVerilog description 17

SystemVerilog Code 17

RISC-V Testbench for program.txt 19

Simulation Results 20

Task 1.5 – Running a test program 21

RISC-V Testbench for test_program.pdf (same testbench as 1.4 but testing with different CPUIn) 21

Simulation Results 23

Task 2.1 – A program to count the number of 1's in an input byte 25

Assembly/Machine Code 25

Terminal 26

Task 2.2 – A program to multiply two bytes using the shift and add method 28

Assembly/Machine Code 28

Terminal 29

Task 1.1

Inst	RegWrite	$ImmSrc_{2:0}$	ALUSrc	$ALUControl_{4:0}$	MemWrite	$ResultSrc_{1:0}$	$PCSrc_{1:0}$
add	1	XXX	0	X0X10	0	01	00
sub	1	XXX	0	X1X10	0	01	00
or	1	XXX	0	XX111	0	01	00
and	1	XXX	0	XX011	0	01	00
sll	1	XXX	0	0XX00	0	01	00
srl	1	XXX	0	1XX00	0	01	00
slt	1	XXX	0	X1X01	0	01	00
addi	1	000	1	X0X10	0	01	00
ori	1	000	1	XX111	0	01	00
andi	1	000	1	XX011	0	01	00
slli	1	000	1	0XX00	0	01	00
srli	1	000	1	1XX00	0	01	00
slti	1	000	1	X1X01	0	01	00
lw	1	000	1	X0X10	0	10	00
sw	0	001	1	X0X10	1	XX	00
beq	0	010	0	X1X10	0	XX	{0, Zero}
bne	0	010	0	X1X10	0	XX	$\{0,\overline{Zero}\}$
blt	0	010	0	X1X10	0	XX	{0,Negative}
bge	0	010	0	X1X10	0	XX	$\{0, \overline{Negative}\}$
jal	1	100	X	XXXXX	0	11	01
jalr	1	000	1	X0X10	0	11	10
lui	1	011	X	XXXXX	0	00	00

Task 1.2 – Writing SystemVerilog description of the sub-blocks

In this task we are required to complete the code for the remaining modules of the RISC-V Microarchitecture (program_counter, control_unit, extend, and alu).

Program counter module

SystemVerilog code

```
module program_counter (output logic [31:0] PC, PCPlus4,
                        input logic [31:0] PCTarget, ALUResult,
                        input logic [1:0] PCSrc,
                        input logic Reset, CLK);

assign PCPlus4 = PC + 4;

always_ff @ (posedge CLK) begin
    if (Reset) begin
        PC <= 32'h00000000;
    end
    else begin
        case (PCSrc)
            2'b00: PC = PCPlus4;
            2'b01: PC = PCTarget;
            //2'b10: PC = ALUResult;
            2'b10: PC = PCTarget;
        endcase
    end
end

endmodule
```

Extend module

SystemVerilog code

```
module extend (output logic [31:0] ImmExt,
               input logic [31:0] Instr,
               input logic [2:0] ImmSrc);

    assign ImmExt =
        (ImmSrc == 3'b000) ? {{20{Instr[31]}}, Instr[31:20]} :  // I-Type
        (ImmSrc == 3'b001) ? {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}
:  // S-Type
        (ImmSrc == 3'b010) ? {{19{Instr[31]}}, Instr[31], Instr[7],
Instr[30:25], Instr[11:8], 1'b0} : // B-Type
        (ImmSrc == 3'b011) ? {Instr[31:12], 12'b0} :  // U-Type
        (ImmSrc == 3'b100) ? {{12{Instr[31]}}, Instr[19:12], Instr[20],
Instr[30:21], 1'b0} : // J-Type
        32'b0;  // Default case

endmodule
```

Control Unit Module

SystemVerilog Code

```
module control_unit (output logic [1:0] PCSrc,
                     output logic [1:0] ResultSrc,
                     output logic MemWrite, ALUSrc, RegWrite,
                     output logic [4:0] ALUControl,
                     output logic [2:0] ImmSrc,
                     input logic [31:0] Instr,
                     input logic Zero, Negative);

logic [6:0] opcode;
logic [2:0] funct3;
logic [6:0] funct7;

assign opcode = Instr[6:0];
assign funct3 = Instr[14:12];
assign funct7 = Instr[31:25];

always_comb begin
    case (opcode)
        7'b0110011: begin
            RegWrite = 1;
            ImmSrc = 3'b???;
            ALUSrc = 0;
```

```
            MemWrite = 0;
            ResultSrc = 2'b01;
            PCSrc = 2'b00;

            case (funct3)
                3'b000: ALUControl = (funct7 == 7'b0100000) ? 5'b?1?10 :
5'b?0?10;
                3'b110: ALUControl = 5'b??111;
                3'b111: ALUControl = 5'b??011;
                3'b001: ALUControl = 5'b0??00;
                3'b101: ALUControl = 5'b1??00;
                3'b010: ALUControl = 5'b?1?01;
            endcase
        end

        7'b0010011: begin
            RegWrite = 1;
            ImmSrc = 3'b000;
            ALUSrc = 1;
            MemWrite = 0;
            ResultSrc = 2'b01;
            PCSrc = 2'b00;

            case (funct3)
                3'b000: ALUControl = 5'b?0?10;
                3'b110: ALUControl = 5'b??111;
                3'b111: ALUControl = 5'b??011;
                3'b001: ALUControl = 5'b0??00;
                3'b101: ALUControl = 5'b1??00;
                3'b010: ALUControl = 5'b?1?01;
            endcase
        end

        7'b0000011: begin
            RegWrite = 1;
            ImmSrc = 3'b000;
            ALUSrc = 1;
            MemWrite = 0;
            ResultSrc = 2'b10;
            PCSrc = 2'b00;
            ALUControl = 5'b?0?10;
        end

        7'b0100011: begin
            RegWrite = 0;
            ImmSrc = 3'b001;
            ALUSrc = 1;
            MemWrite = 1;
```

```
            ResultSrc = 2'b01;
            PCSrc = 2'b00;
            ALUControl = 5'b?0?10;
        end

        7'b1100011: begin
            RegWrite = 0;
            ImmSrc = 3'b010;
            ALUSrc = 0;
            MemWrite = 0;
            ResultSrc = 2'b??;
            ALUControl = 5'b?1?10;

            case (funct3)
                3'b000: PCSrc = (Zero) ? 2'b01 : 2'b00;
                3'b001: PCSrc = (Zero) ? 2'b00 : 2'b01;
                3'b100: PCSrc = (Negative) ? 2'b01 : 2'b00;
                3'b101: PCSrc = (Negative) ? 2'b00 : 2'b01;

            endcase
        end

        7'b1101111: begin
            RegWrite = 1;
            ImmSrc = 3'b100;
            ALUSrc = 0;
            ALUControl = 5'b?????;
            MemWrite = 0;
            ResultSrc = 2'b11;
            PCSrc = 2'b01;
        end

        7'b1100111: begin
            RegWrite = 1;
            ImmSrc = 3'b000;
            ALUSrc = 1;
            ALUControl = 5'b?????;
            MemWrite = 0;
            ResultSrc = 2'b11;
            PCSrc = 2'b10;
        end

        7'b0110111: begin
            RegWrite = 1;
            ImmSrc = 3'b011;
            ALUSrc = 0;
            ALUControl = 5'b?????;
            MemWrite = 0;
```

```
            ResultSrc = 2'b00;
            PCSrc = 2'b00;
        end

    endcase

end

endmodule
```

ALU Module

SystemVerilog Code

```
module alu (output logic signed [31:0] ALUResult,
            output logic Zero, Negative,
            input logic signed [31:0] SrcA, SrcB,
            input logic [4:0] ALUControl);

    logic signed [31:0] ArithResult;
    logic signed [31:0] LogicResult;
    logic signed [31:0] ShiftResult;
    logic signed [31:0] SLTResult;
    logic [1:0] ALUOp;

    assign ALUOp = ALUControl[1:0];

    assign ArithResult = (ALUControl[3]) ?  SrcA - SrcB : SrcA + SrcB;

    assign LogicResult = (ALUControl[2]) ? SrcA | SrcB : SrcA & SrcB;

    assign ShiftResult = (ALUControl[4]) ? SrcA >> SrcB[4:0] : SrcA <<
SrcB[4:0];

    assign SLTResult = (SrcA < SrcB) ? 32'b1 : 32'b0;

    always_comb begin
        case (ALUOp)
            2'b00: ALUResult = ShiftResult;
            2'b01: ALUResult = SLTResult;
            2'b10: ALUResult = ArithResult;
            2'b11: ALUResult = LogicResult;
            default: ALUResult = 32'b0;
        endcase
    end

    assign Zero = (ALUResult == 0);

    assign Negative = (ALUOp == 2'b01) ? ALUResult[0] : 1'b0;

endmodule
```

Task 1.3 – Simulations of sub-blocks using testbenches

Program counter Testbench

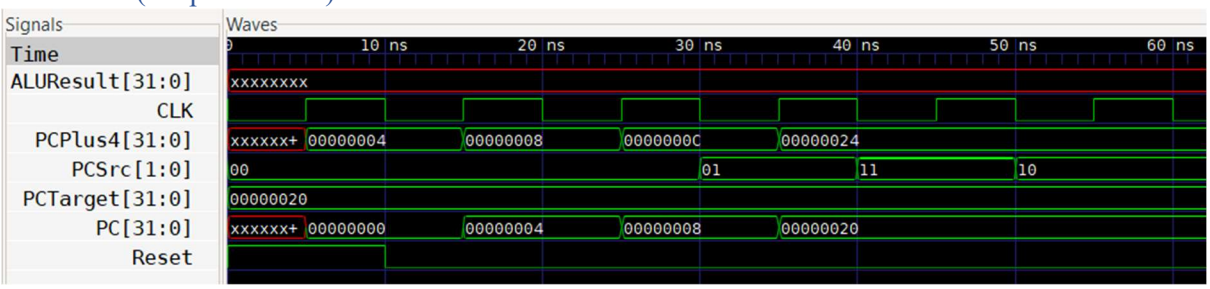
```
`timescale 1ns/1ps
`include "program_counter.sv"

module program_counter_tb;

    logic [31:0] PC, PCPlus4;
    logic [31:0] PCTarget, ALUResult;
    logic [1:0] PCSrc;
    logic Reset, CLK;

    program_counter dut( PC, PCPlus4, PCTarget, ALUResult, PCSrc, Reset, CLK);

    always #5 CLK = ~CLK;

    initial begin
        $dumpfile("program_counter_tb.vcd");
        $dumpvars(0, program_counter_tb);

        CLK = 0;
        Reset = 1;
        PCSrc = 2'b00;
        PCTarget = 32'h00000020;

        #10 Reset = 0;

        #10 PCSrc = 2'b00;

        #10 PCSrc = 2'b01;

        #10 PCSrc = 2'b11;

        #10 PCSrc = 2'b10;
    end

    initial begin
        $monitor("Time = %3d | Reset = %b | PCSrc = %b | PCTarget = %h | PC =
%h \n", $time, Reset, PCSrc, PCTarget, PC);
    end

endmodule
```

Simulation results

Terminal (Text form):

```
Time =  0 | Reset = 1 | PCSrc = 00 | PCTarget = 00000020 | PC = xxxxxxxx

Time =  5 | Reset = 1 | PCSrc = 00 | PCTarget = 00000020 | PC = 00000000

Time = 10 | Reset = 0 | PCSrc = 00 | PCTarget = 00000020 | PC = 00000000

Time = 15 | Reset = 0 | PCSrc = 00 | PCTarget = 00000020 | PC = 00000004

Time = 25 | Reset = 0 | PCSrc = 00 | PCTarget = 00000020 | PC = 00000008

Time = 30 | Reset = 0 | PCSrc = 01 | PCTarget = 00000020 | PC = 00000008

Time = 35 | Reset = 0 | PCSrc = 01 | PCTarget = 00000020 | PC = 00000020

Time = 40 | Reset = 0 | PCSrc = 11 | PCTarget = 00000020 | PC = 00000020

Time = 50 | Reset = 0 | PCSrc = 10 | PCTarget = 00000020 | PC = 00000020
```

GTKWave (Graphical form):



For the first 10 ns (#10) Reset is enable (Reset = 1), therefore, whatever value PCSrc has, PC will be set to 0x00000000. However, after Reset is disabled (Reset = 0) PCSrc is set to 2'b00 for the first 30 ns, therefore PC gets incremented by 4 from 0x0 to 0x4 at the first positive clock edge and from 0x4 to 0x8 at the second positive clock edge. Then PCSrc is set to 2'b11 where PC is now PC = PCTarget. To test the module, PCTarget was set to 0x20. Therefore, at the next positive clock edge PC = 0x20. When PCSrc = 2'b10, PC should equal ALUResult, however, this has been done in the complete interconnected risc_v module that will be explained later. However, in the program counter module PC will also equal PCTarget, therefore, at the next positive clock edge PC will stay at 0x20.

Extend Testbench

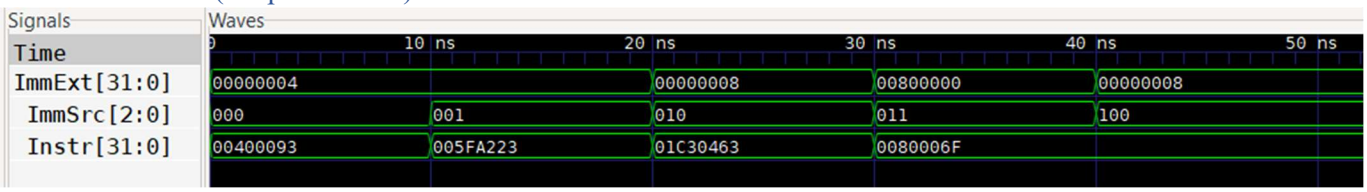
```
`timescale 1ns/1ps
`include "extend.sv"

module extend_tb;
    logic [31:0] ImmExt;
    logic [31:0] Instr;
    logic [2:0] ImmSrc;

    extend dut(ImmExt, Instr, ImmSrc);

    initial begin
        $dumpfile("extend_tb.vcd");
        $dumpvars(0, extend_tb);

        Instr = 32'h00400093;
        ImmSrc = 3'b000;
        #10;

        Instr = 32'h005FA223;
        ImmSrc = 3'b001;
        #10;

        Instr = 32'h01C30463;
        ImmSrc = 3'b010;
        #10;

        Instr = 32'h80000FB7;
        ImmSrc = 3'b011;
        #10;

        Instr = 32'h0080006F;
        ImmSrc = 32'b100;
        #40;
        $finish;
    end

    initial begin
        $monitor("Time = %3d | Instr = %h | ImmSrc = %b | ImmExt = %h", $time,
Instr, ImmSrc, ImmExt);
    end

endmodule
```

Simulation Results

Terminal (Text form)

```
Time =  0 | Instr = 00400093 | ImmSrc = 000 | ImmExt = 00000004

Time =  10 | Instr = 005fa223 | ImmSrc = 001 | ImmExt = 00000004

Time =  20 | Instr = 01c30463 | ImmSrc = 010 | ImmExt = 00000008

Time =  30 | Instr = 80000fb7 | ImmSrc = 011 | ImmExt = 80000000

Time =  40 | Instr = 0080006f | ImmSrc = 100 | ImmExt = 00000008
```

GTKWave (Graphical form)



This module extends the immediate value in the instruction according to the type of instruction. The first instruction (0x00400093) is an I-type instruction, since ImmSrc = 3'b000. The immediate value of the instruction is 0x4, considering the given Table 2. Therefore, this immediate value is extended to a 32-bit immediate by sign extending it, in this case the most significant bit (MSB) of 0x4 is 0. After 10 ns the next instruction inputted is a S-type instruction (0x005FA223). Its assembly code is sw x5, 0x2(x31). Therefore, the immediate is 000000000010. The MSB of the instruction is 0, the Instr[11:7] bits are 0100 and Instr[31:25] are 0000000. Therefore, ImmExt is 0x4, as shown in the terminal and GTKWave. The next instruction (0x01C30463) is a B-type instruction. The assembly code is beq x6, x28, L1. The offset from this instruction to L1 is 0000000001000. Instr[11:8] is 0100, a 0 bit is inputted as the least significant bit (LSB), Instr[30:25] is 000000, Instr[7] is 0 and Instr[31] is 0, therefore, the ImmExt of this instruction is 0x8. The next instruction 0x80000FB7 is a U-type instruction. 12 bits of 0 are inputted from the LSB side of the hexadecimal extended immediate. Then the last 20 bits are the 5 most significant bytes from the instruction (Instr[31:12) 80000, therefore, ImmExt = 0x80000000. Lastly, instruction 0x80000FB7 is a J-type instruction with an assembly code jal x0, 8. 1 bit of 0 is inputted from the RHS, then bits Instr[31:12] the LHS, which are the last 3 bytes 008, Instr[20] is 0, Instr[19:12] is 00000000 and Instr[31] is 0. Therefore, ImmExt = 0x0000008.

Control Unit Testbench

```
`timescale 1ns/1ps
`include "control_unit.sv"

module control_unit_tb;
    logic [1:0] PCSrc;
    logic [1:0] ResultSrc;
    logic MemWrite, ALUSrc, RegWrite;
    logic [4:0] ALUControl;
    logic [2:0] ImmSrc;
    logic [31:0] Instr;
    logic Zero, Negative;
```

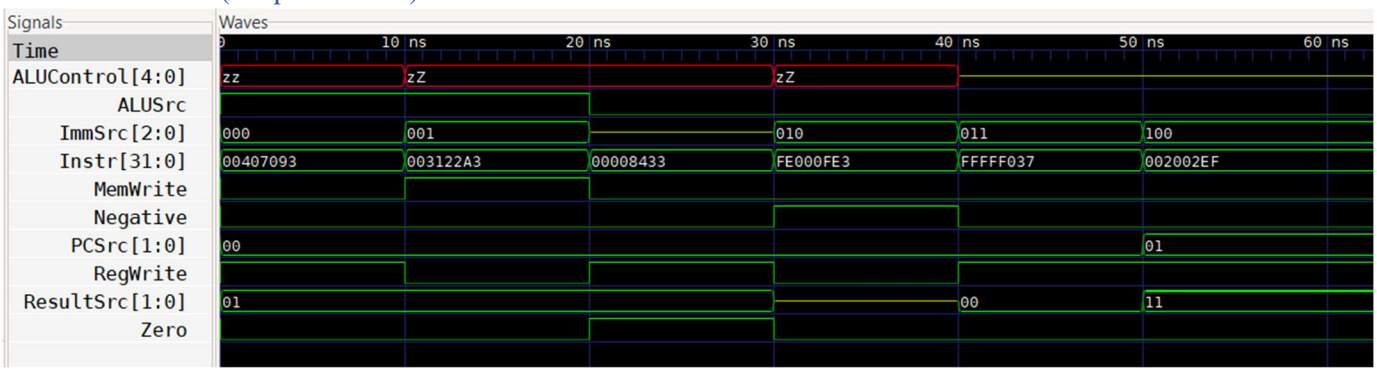
```
    control_unit dut(PCSrc, ResultSrc, MemWrite, ALUSrc, RegWrite, ALUControl,
ImmSrc, Instr, Zero, Negative);

    initial begin
        $dumpfile("control_unit_tb.vcd");
        $dumpvars(0, control_unit_tb);

        Zero = 0;
        Negative = 0;

        Instr = 32'b000000000100_00000_111_00001_0010011;
        #10;

        Instr = 32'b0000000_00011_00010_010_00101_0100011;
        #10;

        Instr = 32'b0000000_00000_00001_000_01000_0110011;
        Zero = 1;
        Negative = 0;
        #10;

        Instr = 32'b1111111_00000_00000_000_1111_1_1100011;
        Zero = 0;
        Negative = 1;
        #10;

        Instr = 32'b11111111111111111111_00000_0110111;
        Zero = 0;
        Negative = 0;
        #10;

        Instr = 32'b0_0000000001_0_00000000_00101_1101111;
        Zero = 0;
        Negative = 0;
        #100;
        $finish;
    end

    initial begin
        $monitor("Time = %3d | Instr = %h | PCSrc = %b | ResultSrc = %b |
MemWrite = %b | ALUSrc = %b | RegWrite = %b | ALUControl = %b | ImmSrc = %b",
$time, Instr, PCSrc, ResultSrc, MemWrite, ALUSrc, RegWrite, ALUControl,
ImmSrc);
    end
endmodule
```

Simulation Results

Terminal (Text form)

```
Time =  0 | Instr = 00407093 | PCSrc = 00 | ResultSrc = 01 | MemWrite = 0 | ALUSrc = 1 | RegWrite = 1 | ALUControl = zz011 | ImmSrc = 000

Time = 10 | Instr = 003122a3 | PCSrc = 00 | ResultSrc = 01 | MemWrite = 1 | ALUSrc = 1 | RegWrite = 0 | ALUControl = z0z10 | ImmSrc = 001

Time = 20 | Instr = 00008433 | PCSrc = 00 | ResultSrc = 01 | MemWrite = 0 | ALUSrc = 0 | RegWrite = 1 | ALUControl = z0z10 | ImmSrc = zzz

Time = 30 | Instr = fe000fe3 | PCSrc = 00 | ResultSrc = zz | MemWrite = 0 | ALUSrc = 0 | RegWrite = 0 | ALUControl = z1z10 | ImmSrc = 010

Time = 40 | Instr = fffff037 | PCSrc = 00 | ResultSrc = 00 | MemWrite = 0 | ALUSrc = 0 | RegWrite = 1 | ALUControl = zzzzz | ImmSrc = 011

Time = 50 | Instr = 002002ef | PCSrc = 01 | ResultSrc = 11 | MemWrite = 0 | ALUSrc = 0 | RegWrite = 1 | ALUControl = zzzzz | ImmSrc = 100
```

GTKWave (Graphical form)



The first instruction is an I-type instruction, therefore, RegWrite = 1, ImmSrc = 000, ALUSrc = 1, MemWrite = 0, ResultSrc = 01, PCSrc = 00. With funct3 = 111, therefore, ALUControl is zz011, as shown in the terminal and GTKWave and the SystemVerilog code. The next instruction is a S-type instruction, therefore, RegWrite = 0, ImmSrc = 001, ALUSrc = 1, MemWrite = 1, ResultSrc = 01, PCSrc = 00, therefore, ALUControl is z0z10, as shown in the terminal and GTKWave and the SystemVerilog code. Instruction 0x00008433 is a R-type, therefore, RegWrite = 1, ImmSrc = zzz, ALUSrc = 0, MemWrite = 0, ResultSrc = 01, PCSrc = 00, therefore, ALUControl is z0z10. The next instruction, 0xFE000FE3, is a B-type instruction. RegWrite = 0, ImmSrc = 010, ALUSrc = 0, MemWrite = 0, ResultSrc = zz, PCSrc = 00, therefore, ALUControl is z1z10.

ALU Testbench

```
`timescale 1ns/1ps
`include "alu.sv"

module alu_tb;
    logic signed [31:0] ALUResult;
    logic Zero, Negative;
    logic signed [31:0] SrcA, SrcB;
    logic [4:0] ALUControl;

    alu dut (ALUResult, Zero, Negative, SrcA, SrcB, ALUControl);

    initial begin
        $dumpfile("alu_tb.vcd");
        $dumpvars(0, alu_tb);

        SrcA = 32'd5;
        SrcB = 32'd5;
        ALUControl = 5'b00010;
        #10;

        SrcA = 32'd5;
        SrcB = 32'd5;
        ALUControl = 5'b01010;
        #10;

        SrcA = 32'hF0F0F0F0;
        SrcB = 32'h0F0F0F0F;
        ALUControl = 5'b00011;
        #10;

        SrcA = 32'hF0F0F0F0;
        SrcB = 32'h0F0F0F0F;
        ALUControl = 5'b00111;
        #10;

        SrcA = 32'd4;
        SrcB = 32'd2;
        ALUControl = 5'b00000;
        #10;

        SrcA = 32'd4;
        SrcB = 32'd2;
        ALUControl = 5'b10000;
        #10;

        SrcA = 32'd5;
        SrcB = 32'd10;
```

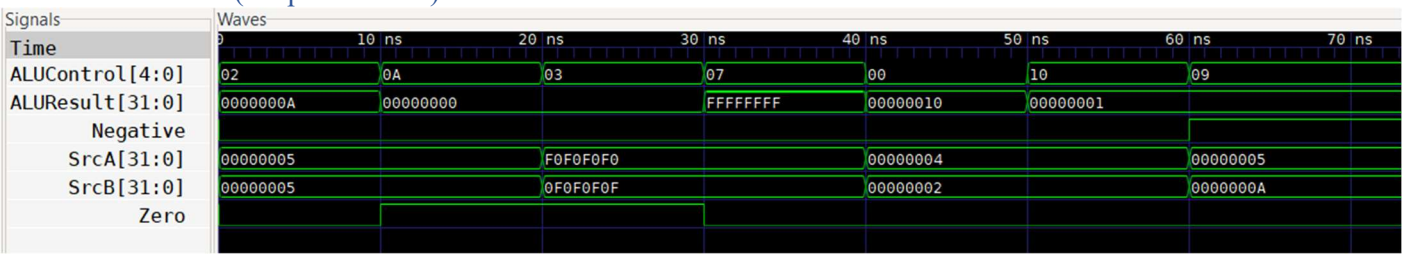
```
        ALUControl = 5'b01001;
        #40;
        $finish;
    end

    initial begin
        $monitor("Time = %3d | SrcA = %h | SrcB = %h | Zero = %b | Negative = %b | ALUControl = %h | ALUResult = %h \n", $time, SrcA, SrcB, Zero, Negative, ALUControl, ALUResult);
    end
endmodule
```

Simulation Results

Terminal (Text form)

```
Time =  0 | SrcA = 00000005 | SrcB = 00000005 | Zero = 0 | Negative = 0 | ALUControl = 02 | ALUResult = 0000000a

Time = 10 | SrcA = 00000005 | SrcB = 00000005 | Zero = 1 | Negative = 0 | ALUControl = 0a | ALUResult = 00000000

Time = 20 | SrcA = f0f0f0f0 | SrcB = 0f0f0f0f | Zero = 1 | Negative = 0 | ALUControl = 03 | ALUResult = 00000000

Time = 30 | SrcA = f0f0f0f0 | SrcB = 0f0f0f0f | Zero = 0 | Negative = 0 | ALUControl = 07 | ALUResult = ffffffff

Time = 40 | SrcA = 00000004 | SrcB = 00000002 | Zero = 0 | Negative = 0 | ALUControl = 00 | ALUResult = 00000010

Time = 50 | SrcA = 00000004 | SrcB = 00000002 | Zero = 0 | Negative = 0 | ALUControl = 10 | ALUResult = 00000001

Time = 60 | SrcA = 00000005 | SrcB = 0000000a | Zero = 0 | Negative = 1 | ALUControl = 09 | ALUResult = 00000001
```

GTKWave (Graphical Form)



The two inputs to the ALU are SrcA and SrcB. For the first set of inputs, ALUControl = 02 (00010) suggests adding the two values together. Therefore, 00000005 + 00000005 = 0000000A = ALUResult. Considering ALUResult != 00000000 Zero = 0 and since the MSB of the signed ALUResult isn't 1

Negative = 0. Next, with the same inputs, however, ALUControl = 0A (01010) suggesting subtraction ALUResult = 00000000, therefore, Zero = 1 and Negative = 0. Next, SrcA = F0F0F0F0 and SrcB = 0F0F0F0F. The ALUControl = 00011, therefore, SrcA and SrcB are and'ed together. Therefore, ALUResult = 00000000, so Zero = 1 and Negative = 0. With the same inputs but ALUControl = 00111, the two inputs are now or'ed together, meaning ALUResult = FFFFFFFF. Considering the MSB = 1, and ALUResult != 00000000, Zero = 0 and Negative = 1. Next, SrcA = 00000004 AND SrcB = 00000002, with ALUControl = 00000. This ALUControl shifts the SrcA by SrcB[4:0] amount to the left, therefore, ALUResult = 00000010. Now with the same inputs we will shift SrcA by SrcB[4:0] amount to the right, since ALUControl = 10000. Therefore, ALUResult = 00000001. Lastly, to check the set less than function of the ALU module, we used inputs SrcA = 00000005 and SrcB = 0000000A, with ALUControl = 01001. This function checks whether SrcA is less than SrcB by subtracting the two. Then the MSB of the result is zero extended. If SrcA < SrcB the MSB of the result is 1, this is then zero extended, therefore, ALUResult = 00000001 and Negative = 1, suggesting that SrcA is less than SrcB and the SLT function is working correctly.

Task 1.4 – Completing top-level RISC-V SystemVerilog description

Having completed each sub-block of the RISC-V microarchitecture individually, we will now interconnect them together to form risc_v module.

SystemVerilog Code

```
`include "instruction_memory.sv"
`include "reg_file.sv"
`include "extend.sv"
`include "alu.sv"
`include "data_memory_and_io.sv"
`include "program_counter.sv"
`include "control_unit.sv"

module risc_v(output logic [31:0] CPUOut,
                input logic [31:0] CPUIn,
                input logic Reset, CLK);

    logic [31:0] Instr, WD3, RD1, RD2, SrcA, SrcB, ALUResult, WD, RD, A,
Result, ImmExt, PCTarget, PCNext, PC, PCPlus4;
    logic [31:0] BranchTarget, JalTarget, JalrTarget;
    logic [4:0] A1, A2, A3;
    logic MemWrite, ALUSrc, RegWrite;
    logic Zero, Negative;
    logic [4:0] ALUControl;
    logic [2:0] ImmSrc;
    logic [1:0] ResultSrc;
    logic [1:0] PCSrc;
    logic WE3, WE;

    assign BranchTarget = PC + ImmExt;
    assign JalTarget = PC + ImmExt;
    assign JalrTarget = ALUResult;
```

```
    assign PCTarget = (PCSrc == 2'b01) ? BranchTarget :
                      (PCSrc == 2'b10) ?
                      ((Instr[6:0] == 7'b1101111) ? JalTarget :
                       (Instr[6:0] == 7'b1100111) ? JalrTarget : 32'hXXXXXXXX)
                  : 32'hXXXXXXXX;

    program_counter PC_module(PC, PCPlus4, PCTarget, ALUResult, PCSrc, Reset,
CLK);

    instruction_memory IM_module(Instr, PC);

    control_unit CU_module(PCSrc, ResultSrc, MemWrite, ALUSrc, RegWrite,
ALUControl, ImmSrc, Instr, Zero, Negative);

    assign A1 = Instr[19:15];
    assign A2 = Instr[24:20];
    assign A3 = Instr[11:7];

    assign WE3 = RegWrite;

    reg_file RF_module(RD1, RD2, WD3, A1, A2, A3, WE3, CLK);

    extend EXT_module(ImmExt, Instr, ImmSrc);

    assign SrcA = RD1;
    assign SrcB = (ALUSrc) ? ImmExt : RD2;

    alu ALU_module(ALUResult, Zero, Negative, SrcA, SrcB, ALUControl);

    assign A = ALUResult;
    assign WD = RD2;
    assign WE = MemWrite;

    data_memory_and_io DM_module(RD, CPUOut, A, WD, CPUIn, WE, CLK);

    assign Result = (ResultSrc == 2'b00) ? ImmExt :
                    (ResultSrc == 2'b01) ? ALUResult :
                    (ResultSrc == 2'b10) ? RD :
                    PCPlus4;

    assign WD3 = Result;

endmodule
```

RISC-V Testbench for program.txt

```
`timescale 1ns/1ps
`include "risc_v.sv"

module risc_v_tb;
    logic [31:0] CPUOut, CPUIn;
    logic Reset, CLK;

    risc_v dut(CPUOut, CPUIn, Reset, CLK);

    initial begin // Generate clock signal with 20 ns period
        CLK = 0;
        forever #10 CLK = ~CLK;
    end

    initial begin // Apply stimulus
        $dumpfile("risc_v_tb.vcd");
        $dumpvars(0, risc_v_tb);

        Reset = 1;    // Initial reset
        CPUIn = 32'd0; // Default CPUIn value

        #20;
        Reset = 0;    // Deassert reset to begin execution

        // Simulate the external input changing (CPUIn changes when address
0x7FFFFFC is read)
        #20 CPUIn = 32'h00000005;

        #1260;
        $finish;
    end

    always @ (negedge CLK)
        $display ("t = %3d | CPUIn = %h | CPUOut = %h | Reset = %b | PCSrc =
%b | PC = %h | PCTarget = %h | ImmExt = %h | Instr = %h | ALUResult = %h",
$time, CPUIn, CPUOut, Reset, dut.PCSrc, dut.PC, dut.PCTarget, dut.ImmExt,
dut.Instr, dut.ALUResult);

endmodule
```

Simulation Results

Terminal (Text form)

```
t =  20 | CPUIn = 00000000 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000000 | PCTarget = xxxxxxxx | ImmExt = 00000032 | Instr = 03200093 | ALUResult = 00000032

t =  40 | CPUIn = 00000005 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000004 | PCTarget = xxxxxxxx | ImmExt = 80000000 | Instr = 80000537 | ALUResult = 00000000

t =  60 | CPUIn = 00000005 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000008 | PCTarget = xxxxxxxx | ImmExt = fffffffc | Instr = ffc52103 | ALUResult = 7ffffffc

t =  80 | CPUIn = 00000005 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000000c | PCTarget = xxxxxxxx | ImmExt = X0X0XX0X | Instr = 402081b3 | ALUResult = 0000002d

t = 100 | CPUIn = 00000005 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000010 | PCTarget = xxxxxxxx | ImmExt = fffffffc | Instr = fe352e23 | ALUResult = 7ffffffc

t = 120 | CPUIn = 00000005 | CPUOut = 0000002d | Reset = 0 | PCSrc = 01 | PC = 00000014 | PCTarget = 00000014 | ImmExt = 00000000 | Instr = 00000063 | ALUResult = 00000000
```

Firstly, we will convert the instructions from machine code to assembly code, to understand whether the module is working correctly by expecting the result of the CPUOut. The first instruction is addi x1, x0, 0x32. Which saves 0x0 + 0x32 in register x1. The next instruction (0x80000537) is a lui x9, 0x80000 which puts the immediate 0x80000 in the upper 20 bits of register x9 and 0's in the lower 20 bits, resulting in the ImmExt = 0x80000000. As seen in the testbench of this module CPUIn = 0x00000005 at 40 ns. Then instruction FFC52103 is a lw x2, -4(x9) which adds base address x10 to the offset -4, x2 holds the data value at address x9 – 4. In two's complement the offset -4 is 0xFFFFFFFC. Therefore, 0x80000000 + 0xFFFFFFFC = 0x7FFFFFFC, as shown in ALUResult. This means we are loading a word from memory address 0x7FFFFFFC to register x2. Since CPUIN = 0x00000005, then x2 = CPUIn. The next instruction is 402081B3. The assembly code is sub x3, x1, x2, this instruction saves the result of x1 – x2 to x3. Therefore, 0x32 – 0x5 is saved in x3. As shown in ALUResult the result is 0x2D. Then instruction FE352E23 does sw x3, -4(x9) which performs memory write to address base register (x9) + offset (-4) which was already calculated as 0x7FFFFFFC which is also shown in the ALUResult. This stores the value from register x3 into memory at 0x7FFFFFFC. 0x7FFFFFFC is a memory-mapped I/O register that directly updates CPUOut. Therefore, once the last instruction is executed, which is a beq x0, x0, 0, ultimately branching to itself, effectively creating an infinite loop, CPUOut gets updated to 0x2D which is the result we were looking for.

Task 1.5 – Running a test program

RISC-V Test Program

Memory address	Machine code	Assembly code
0x00000000	80000fb7	lui x31, 0x80000 # Memory-mapped address
0x00000004	ffcf8f93	addi x31, x31, 0xFFC # of CPUOut is 0x7FFFFFFC
0x00000008	80000093	addi x1, zero, 0x800 # Test ADDI
0x0000000c	001fa023	sw x1, 0x0(x31) # Output result
0x00000010	00100093	addi x1, zero, 1
0x00000014	00300113	addi x2, zero, 3
0x00000018	002081b3	add x3, x1, x2 # Test ADD
0x0000001c	003fa023	sw x3, 0x0(x31)
0x00000020	402081b3	sub x3, x1, x2 # Test SUB
0x00000024	003fa023	sw x3, 0x0(x31)
0x00000028	0020e1b3	or x3, x1, x2 # Test OR
0x0000002c	003fa023	sw x3, 0x0(x31)
0x00000030	002091b3	sll x3, x1, x2 # Test SLL
0x00000034	003fa023	sw x3, 0x0(x31)
0x00000038	0020a1b3	slt x3, x1, x2 # Test SLT
0x0000003c	003fa023	sw x3, 0x0(x31)
0x00000040	00717193	andi x3, x2, 0x007 # Test ANDI
0x00000044	003fa023	sw x3, 0x0(x31)
0x00000048	00115193	srli x3, x2, 1 # Test SRLI
0x0000004c	003fa023	sw x3, 0x0(x31)
0x00000050	fff0a193	slti x3, x1, 0xFFF # Test SLTI
0x00000054	003fa023	sw x3, 0x0(x31)
0x00000058	00202023	sw x2, 0x0(zero)
0x0000005c	00002183	lw x3, 0x0(zero) # Test LW
0x00000060	003fa023	sw x3, 0x0(x31)
		label1:
0x00000064	00208063	beq x1, x2, label1 # Test BEQ
0x00000068	00209663	bne x1, x2, label2 # Test BNE
0x0000006c	12300093	addi x1, zero, 0x123
0x00000070	003fa023	sw x3, 0x0(x31)
		label2:
0x00000074	00114063	blt x2, x1, label2 # Test BLT
0x00000078	00115663	bge x2, x1, label3 # Test BGE
0x0000007c	45600093	addi x1, zero, 0x456
0x00000080	003fa023	sw x3, 0x0(x31)
		label3:
0x00000084	0100026f	jal x4, 16 # Test JAL
0x00000088	78900193	addi x3, zero, 0x789
0x0000008c	003fa023	sw x3, 0x0(x31)
0x00000090	0100006f	jal zero, finish
0x00000094	11100193	addi x3, zero, 0x111
0x00000098	003fa023	sw x3, 0x0(x31)
0x0000009c	00020067	jalr zero, x4, 0 # Test JALR
		finish:

RISC-V Testbench for test_program.pdf (same testbench as 1.4 but testing with different CPUIn)

```
`timescale 1ns/1ps
`include "risc_v.sv"

module risc_v_tb;
    logic [31:0] CPUOut, CPUIn;
    logic Reset, CLK;

    risc_v dut(CPUOut, CPUIn, Reset, CLK);

    initial begin // Generate clock signal with 20 ns period
        CLK = 0;
        forever #10 CLK = ~CLK;
    end
```

```
    initial begin // Apply stimulus
        $dumpfile("risc_v_tb.vcd");
        $dumpvars(0, risc_v_tb);

        Reset = 1;    // Initial reset
        CPUIn = 32'd0; // Default CPUIn value

        #20;
        Reset = 0;    // Deassert reset to begin execution

        // Simulate the external input changing (CPUIn changes when address
0x7FFFFFFC is read)
        #20 CPUIn = 32'h000000FF;

        #1260;
        $finish;
    end

    always @ (negedge CLK)
        $display ("t = %3d | CPUIn = %h | CPUOut = %h | Reset = %b | PCSrc =
%b | PC = %h | PCTarget = %h | ImmExt = %h | Instr = %h | ALUResult = %h",
$time, CPUIn, CPUOut, Reset, dut.PCSrc, dut.PC, dut.PCTarget, dut.ImmExt,
dut.Instr, dut.ALUResult);

endmodule
```

Simulation Results

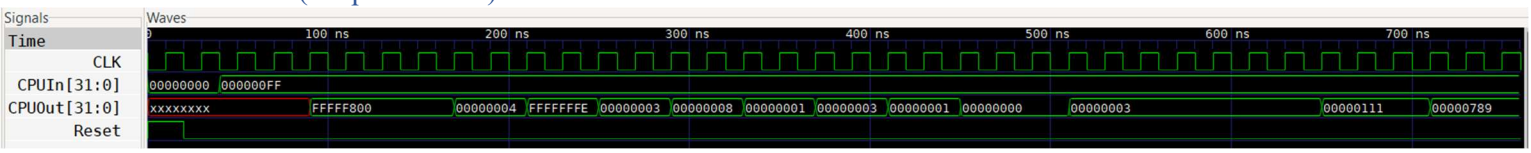
Terminal (Text form)

```
t =  20 | CPUIn = 00000000 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000000 | PCTarget = xxxxxxxx | ImmExt = 80000000 | Instr = 80000fb7 | ALUResult = 00000000
t =  40 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000004 | PCTarget = xxxxxxxx | ImmExt = fffffffc | Instr = ffcf8f93 | ALUResult = 7ffffffc
t =  60 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000008 | PCTarget = xxxxxxxx | ImmExt = fffff800 | Instr = 80000093 | ALUResult = fffff800
t =  80 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000000c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 001fa023 | ALUResult = 7ffffffc
t = 100 | CPUIn = 000000ff | CPUOut = fffff800 | Reset = 0 | PCSrc = 00 | PC = 00000010 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00100093 | ALUResult = 00000001
t = 120 | CPUIn = 000000ff | CPUOut = fffff800 | Reset = 0 | PCSrc = 00 | PC = 00000014 | PCTarget = xxxxxxxx | ImmExt = 00000003 | Instr = 00300113 | ALUResult = 00000003
t = 140 | CPUIn = 000000ff | CPUOut = fffff800 | Reset = 0 | PCSrc = 00 | PC = 00000018 | PCTarget = xxxxxxxx | ImmExt = 00X0XX0X | Instr = 002081b3 | ALUResult = 00000004
t = 160 | CPUIn = 000000ff | CPUOut = fffff800 | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 180 | CPUIn = 000000ff | CPUOut = 00000004 | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = X0X0XX0X | Instr = 402081b3 | ALUResult = fffffffe
t = 200 | CPUIn = 000000ff | CPUOut = 00000004 | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 220 | CPUIn = 000000ff | CPUOut = fffffffe | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00X0XX0X | Instr = 0020e1b3 | ALUResult = 00000003
t = 240 | CPUIn = 000000ff | CPUOut = fffffffe | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 260 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00X0XX0X | Instr = 002091b3 | ALUResult = 00000008
```

```
t = 280 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 300 | CPUIn = 000000ff | CPUOut = 00000008 | Reset = 0 | PCSrc = 00 | PC = 00000038 | PCTarget = xxxxxxxx | ImmExt = 00X0XX0X | Instr = 0020a1b3 | ALUResult = 00000001
t = 320 | CPUIn = 000000ff | CPUOut = 00000008 | Reset = 0 | PCSrc = 00 | PC = 0000003c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 340 | CPUIn = 000000ff | CPUOut = 00000001 | Reset = 0 | PCSrc = 00 | PC = 00000040 | PCTarget = xxxxxxxx | ImmExt = 00000007 | Instr = 00717193 | ALUResult = 00000003
t = 360 | CPUIn = 000000ff | CPUOut = 00000001 | Reset = 0 | PCSrc = 00 | PC = 00000044 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 380 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000048 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00115193 | ALUResult = 00000001
t = 400 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 0000004c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 420 | CPUIn = 000000ff | CPUOut = 00000001 | Reset = 0 | PCSrc = 00 | PC = 00000050 | PCTarget = xxxxxxxx | ImmExt = ffffffff | Instr = fff0a193 | ALUResult = 00000000
t = 440 | CPUIn = 000000ff | CPUOut = 00000001 | Reset = 0 | PCSrc = 00 | PC = 00000054 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 460 | CPUIn = 000000ff | CPUOut = 00000000 | Reset = 0 | PCSrc = 00 | PC = 00000058 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00202023 | ALUResult = 00000000
t = 480 | CPUIn = 000000ff | CPUOut = 00000000 | Reset = 0 | PCSrc = 00 | PC = 0000005c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00002183 | ALUResult = 00000000
t = 500 | CPUIn = 000000ff | CPUOut = 00000000 | Reset = 0 | PCSrc = 00 | PC = 00000060 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc
t = 520 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000064 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00208063 | ALUResult = fffffffe
```

```
t = 540 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 01 | PC = 00000068 | PCTarget = 00000074 | ImmExt = 0000000c | Instr = 00209663 | ALUResult = fffffffe

t = 560 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000074 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00114063 | ALUResult = 00000002

t = 580 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 01 | PC = 00000078 | PCTarget = 00000084 | ImmExt = 0000000c | Instr = 00115663 | ALUResult = 00000002

t = 600 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 01 | PC = 00000084 | PCTarget = 00000094 | ImmExt = 00000010 | Instr = 0100026f | ALUResult = 00000000

t = 620 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000094 | PCTarget = xxxxxxxx | ImmExt = 00000111 | Instr = 11100193 | ALUResult = 00000111

t = 640 | CPUIn = 000000ff | CPUOut = 00000003 | Reset = 0 | PCSrc = 00 | PC = 00000098 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc

t = 660 | CPUIn = 000000ff | CPUOut = 00000111 | Reset = 0 | PCSrc = 10 | PC = 0000009c | PCTarget = 00000088 | ImmExt = 00000000 | Instr = 00020067 | ALUResult = 00000088

t = 680 | CPUIn = 000000ff | CPUOut = 00000111 | Reset = 0 | PCSrc = 00 | PC = 00000088 | PCTarget = xxxxxxxx | ImmExt = 00000789 | Instr = 78900193 | ALUResult = 00000789

t = 700 | CPUIn = 000000ff | CPUOut = 00000111 | Reset = 0 | PCSrc = 00 | PC = 0000008c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 003fa023 | ALUResult = 7ffffffc

t = 720 | CPUIn = 000000ff | CPUOut = 00000789 | Reset = 0 | PCSrc = 01 | PC = 00000090 | PCTarget = 000000a0 | ImmExt = 00000010 | Instr = 0100006f | ALUResult = 00000000

t = 740 | CPUIn = 000000ff | CPUOut = 00000789 | Reset = 0 | PCSrc = 01 | PC = 000000a0 | PCTarget = xxxxxxxx | ImmExt = xxxxxxxX | Instr = xxxxxxxx | ALUResult = 00000000
```

GTKWave (Graphical form)



The first and second line of code sets x31 to 0x7FFFFFFC, a memory-mapped address used for CPUOut. Then 0x800 is loaded into x1, which is then stored at 0x7FFFFFFC, outputting the result to CPUOut. This repeats many times to see the output get updated at CPUOut. Then we initialize x1 = 1 and x2 = 3. Then x3 = x1 + x2 = 4, which is stored in CPUOut, as shown in the simulation results at 170 ns. Then x3 is set to x1 – x2 = 1 – 3 = -2 which is stored in CPUOut. -2 in two's complement is 0XFFFFFFFE. Then x3 is set to the result of x1 being shifted to the left by x2 amount (1 << 3 = 8). This is stored at CPUOut. x3 is then set to the result of whether x1 is less than x2 ((x1 < x2) ? 1 : 0). x1 is infact less than x2 (1 < 3), therefore, x3 is set to 1 which is stored in CPUOut. Next, immediate operations are tested. x3 is set to x2 & 0x7 (3 & 7 = 3), which is stored in CPUOut. x2 is then shifted to the right by the immediate value amount and inputted into x3 (x3 = x2 >> 1 = 3 >> 1 = 1) and stored in CPUOut again. Then x3 is set to the result of whether x1 is less than 0xFFF ((x1 < 0xFFF) ? 1 : 0) x1 is infact less than 0xFFF, therefore, 1 is inputted in x3 and stored in CPUOut. Now the memory operations are tested. x2, which is 3, is stored at memory address 0, then word from memory address 0 is loaded into x3 and lastly the value in register x3 is stored in CPUOut. Now for the conditional executions, if x1 == x2 then a infinite loop on this instruction is executed, however, x1 is not equal to x2 (1 != 3), therefore, it does not loop and the next instruction is executed. If x1 != x2 the program must branch to label2. x1 is not equal to x2, therefore, the program branches to blt x2, x1, label2 (00114063), as shown in the terminal. Now, if x2 < x1, the program branches to label2, executing an infinite loop, however, x2 is not less than x1 (3 > 1), therefore, the next instruction in order is executed. If x2 >= x1, the program branches to label3, this is true, therefore, the next instruction that will be executed is jal x4, 16 (0100026F). The program jumps 16 bytes forwards and stores the return address in x4, therefore, 0x88 is stored in x4. The next instruction that is executed is addi x3, zero, 0x111 (11100193) as shown in the terminal. 0x111 is inputted into x3 and stored in CPUOut, as shown in the terminal and the GTKWave at t = 660 ns. Next instruction jalr zero, x4, 0 is executed where the program jumps back to address in x4 which was 0x88. Therefore, instruction addi x3, zero, 0x789 (78900193) which inputs 0x789 in x3 and is then stored in CPUOut, as shown at t = 720 ns in the terminal and GTKWave. Lastly, next instruction in order, jal zero, finish is executed which is an unconditional jump to 'finish' which is the end of the program.

Task 2.1 – A program to count the number of 1's in an input byte

Assembly/Machine Code

```
lui x31, 0x80000     // 80000fb7
addi x31, x31, 0xFFC // ffcf8f93
lw x1, 0x0(x31)      // 000fa083


addi x7, x0, 8       // 00800393
addi x5, x0, 0       // 00000293
addi x6, x0, 0       // 00000313
loop:
andi x28, x1, 1       // 0010fe13
beq x28, x0, skip     // 000e0463
addi x5, x5, 1       // 00128293
skip:
srli x1, x1, 1       // 0010d093
addi x6, x6, 1       // 00130313
beq x6, x7, end loop // 00730463
jal x0, loop         // fe9ff06f
end loop:
sw x5, 0x0(x31)      // 005fa023
```

The first to the fourth line of instructions are responsible for storing and loading from and to data memory address 0x7FFFFFFC to output to CPUOut and input to CPUIn. In this case, we are storing x1 (0xFF, the CPUIn which will be tested to see how many 1's it contains) in CPUOut. Then we initiate x7 = 8 (limit to the increment of i, to count the number of 1's in the least significant byte), x5 = 0 (sum) and x6 = 0 (i). Firstly, the least significant bit of the CPUIn is tested, andi x28, x1, 1 suggests that if the least significant bit of x1 is 1 then x1 & 1 = 1 = x28. Then instruction beq x28, x0, skip suggests that if x28 == x0 = 0, suggesting that the LSB was 0, the program branches to test the next bit by shifting x1 to the right by 1 and incrementing i up by 1. If i is not yet equal to 8 then the program jumps back to the loop and tests the least significant bit of the shifted x1. However, if x28 != 0, suggesting that the LSB was 1, then beq x28, x0, skip does not branch to skip but instead inputs to x5 the result of x5 + 1, this result is the current count of the number of 1's in the input byte. This process is repeated until i = 8.

Once i is incremented up to 8, beq x6, x7, end loop is executed x6 will equal to x7, therefore, the program will branch to end loop where the result of number of 1's (the value of x5) is stored in CPUOut.

Terminal

```
t =  20 | CPUIn = 00000000 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000000 | PCTarget = xxxxxxxx | ImmExt = 80000000 | Instr = 80000fb7 | ALUResult = 00000000

t =  40 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000004 | PCTarget = xxxxxxxx | ImmExt = fffffffc | Instr = ffcf8f93 | ALUResult = 7fffffffc

t =  60 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000008 | PCTarget = xxxxxxxx | ImmExt = 000000ff | Instr = 0ff00093 | ALUResult = 000000ff

t =  80 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000000c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 001fa023 | ALUResult = 7fffffffc

t = 100 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000010 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00800393 | ALUResult = 00000008

t = 120 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000014 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00000293 | ALUResult = 00000000

t = 140 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000018 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00000313 | ALUResult = 00000000

t = 160 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 180 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 200 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000001

t = 220 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 0000007f

t = 240 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000001

t = 260 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffff9

t = 280 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 300 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 320 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001
```

```
t = 340 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000002

t = 360 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 0000003f

t = 380 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000002

t = 400 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffffa

t = 420 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 440 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 460 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 480 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000003

t = 500 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 0000001f

t = 520 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000003

t = 540 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffffb

t = 560 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 580 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 600 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 620 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000004

t = 640 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 0000000f
```

t = 660 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000004

t = 680 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffffc

t = 700 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 720 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 740 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 760 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000005

t = 780 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 00000007

t = 800 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000005

t = 820 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffffd

t = 840 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 860 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 880 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 900 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000006

t = 920 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 00000003

t = 940 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000006

t = 960 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = fffffffe

t = 980 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 1000 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 1020 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 1040 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000007

t = 1060 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 00000001

t = 1080 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000007

t = 1100 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00730463 | ALUResult = ffffffff

t = 1120 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000001c | ImmExt = ffffffe8 | Instr = fe9ff06f | ALUResult = 00000000

t = 1140 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010fe13 | ALUResult = 00000001

t = 1160 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 000e0463 | ALUResult = 00000001

t = 1180 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00128293 | ALUResult = 00000008

t = 1200 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0010d093 | ALUResult = 00000000

t = 1220 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000008

t = 1240 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 01 | PC = 00000030 | PCTarget = 00000038 | ImmExt = 00000008 | Instr = 00730463 | ALUResult = 00000000

t = 1260 | CPUIn = 000000ff | CPUOut = 000000ff | Reset = 0 | PCSrc = 00 | PC = 00000038 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 005fa023 | ALUResult = 7ffffffc

t = 1280 | CPUIn = 000000ff | CPUOut = 00000008 | Reset = 0 | PCSrc = 00 | PC = 0000003c | PCTarget = xxxxxxxx | ImmExt = xxxxxxxx | Instr = xxxxxxxx | ALUResult = xxxxxxxx

The program is working correctly because CPUIn = 0xFF contains 8 1's in the least significant bytes, therefore, CPUOut being 0x8 is correct.

Task 2.2 – A program to multiply two bytes using the shift and add method

Assembly/Machine Code

```
lui x31, 0x80000      //80000fb7
addi x31, x31, 0xffc  //ffcf8f93
addi x8, x0, 3        //00300413
addi x9, x0, 4        //00400493
addi x5, x0, 0        //00000293
addi x6, x0, 0        //00000313
addi x28, x0, 8       //00800e13
loop:
andi x7, x9, 1        //0014f393
beq x7, x0, skip      //00038463
add x5, x5, x8        //008282b3
skip:
srli x9, x9, 1        //0014d493
slli x8, x8, 1        //00141413
addi x6, x6, 1        //00130313
beq x6, x28, end loop //01c30463
jal x0, loop          //fffff06f
end loop:
sw x5, 0x0(x31)       //005fa023
```

The first two lines of instructions are responsible for storing and loading from and to data memory address 0x7FFFFFFC to output to CPUOut and input to CPUIn. Then we initiate the multiplicand (x8 = 3), the multiplier (x9 = 4), the product (x5 = 0) and i (counter) (x6 = 0). Then the limit to the i incrementing (x28 = 8). Then we check whether the LSB of the multiplicand is 1 by using the instruction andi x7, x9, 1 the result of x9 & 1 is inputted to x7. Then if the LSB was 0 then beq x7, x0, skip branches to skip. The multiplicand is shifted to the left and the multiplier is shifted to the right, this helps in appending zeros before summing up the partial products (place value shifting). If LSB was 1, then beq x7, x0, skip does not cause a branching to skip, therefore, add x5, x5, x8 is executed. This is responsible for adding up the partial products and multiplying the multiplicand with the current multiplier bit (in binary this is simply the value of the multiplicand), and then the multiplicand and multiplier are shifted. Once addi x6, x6, 1 cause i = 8, beq x6, x28, end loop will get executed successfully and the program branches to end loop. The value of the product (x5) is then stored in CPUOut.

Terminal

```
t =  20 | CPUIn = 00000000 | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000000 | PCTarget = xxxxxxxx | ImmExt = 80000000 | Instr = 80000fb7 | ALUResult = 00000000

t =  40 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000004 | PCTarget = xxxxxxxx | ImmExt = fffffffc | Instr = ffcf8f93 | ALUResult = 7ffffffc

t =  60 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000008 | PCTarget = xxxxxxxx | ImmExt = 00000003 | Instr = 00300413 | ALUResult = 00000003

t =  80 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000000c | PCTarget = xxxxxxxx | ImmExt = 00000004 | Instr = 00400493 | ALUResult = 00000004

t = 100 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000010 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00000293 | ALUResult = 00000000

t = 120 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000014 | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 00000313 | ALUResult = 00000000

t = 140 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000018 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00800e13 | ALUResult = 00000008

t = 160 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 180 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 200 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000002

t = 220 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000006

t = 240 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000001

t = 260 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffff9
```

```
t = 280 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 300 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 320 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 340 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000001

t = 360 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 0000000c

t = 380 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000002

t = 400 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffffa

t = 420 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 440 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000001

t = 460 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000020 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000001

t = 480 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000024 | PCTarget = xxxxxxxx | ImmExt = 00XXXX0X | Instr = 008282b3 | ALUResult = 0000000c

t = 500 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 520 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000018
```

t = 540 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000003

t = 560 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffffb

t = 580 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 600 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 620 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 640 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 660 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000030

t = 680 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000004

t = 700 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffffc

t = 720 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 740 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 760 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 780 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 800 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000060

t = 820 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000005

t = 840 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffffd

t = 860 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 880 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 900 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 920 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 940 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 000000c0

t = 960 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000006

t = 980 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = fffffffe

t = 1000 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 1020 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 1040 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 1060 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 1080 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000180

t = 1100 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000007

t = 1120 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000034 | PCTarget = xxxxxxxx | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = ffffffff

t = 1140 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000038 | PCTarget = 0000001c | ImmExt = ffffffe4 | Instr = fe5ff06f | ALUResult = 00000000

t = 1160 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000001c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014f393 | ALUResult = 00000000

t = 1180 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000020 | PCTarget = 00000028 | ImmExt = 00000008 | Instr = 00038463 | ALUResult = 00000000

t = 1200 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000028 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 0014d493 | ALUResult = 00000000

t = 1220 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000002c | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00141413 | ALUResult = 00000300

t = 1240 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 00000030 | PCTarget = xxxxxxxx | ImmExt = 00000001 | Instr = 00130313 | ALUResult = 00000008

t = 1260 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 01 | PC = 00000034 | PCTarget = 0000003c | ImmExt = 00000008 | Instr = 01c30463 | ALUResult = 00000000

t = 1280 | CPUIn = 000000ff | CPUOut = xxxxxxxx | Reset = 0 | PCSrc = 00 | PC = 0000003c | PCTarget = xxxxxxxx | ImmExt = 00000000 | Instr = 005fa023 | ALUResult = 7fffffffc

t = 1300 | CPUIn = 000000ff | CPUOut = 0000000c | Reset = 0 | PCSrc = 00 | PC = 00000040 | PCTarget = xxxxxxxx | ImmExt = xxxxxxxx | Instr = xxxxxxxx | ALUResult = xxxxxxxx

The program is working correctly because the output is CPUOut = 0xC. The two inputs were 0x4 and 0x3, therefore, the answer was 0xC.